

M8 — Student dataclass + roster persistence

Phase: 5 — Roster **Status:** DONE (2026-05-26) **Blocked by:** M2 **Blocks:** M9 **Estimated effort:** 1 day · **Actual:** 0.5 day

1. Purpose

Make student rostering survive server restarts. M2 introduced the `Student` dataclass and the in-memory `ControlRoom.students` dict; M1 added the `student` table to the schema. M8 connects them — every `add_student` writes through to the DB, every `assign_student` updates the row, and a `rehydrate_students_from_db()` helper reattaches a room's roster on demand (M9's join flow will call this when a student lands on the join page after a restart).

Without M8, a server restart silently dropped every roster assignment — a half-finished cohort debrief had no idea which student went to which encounter. M8 closes that gap.

2. Structure

Files touched:

- `portal/ehr_db.py` — appends a "V7 — Student roster (M8)" section with seven CRUD helpers and an `_mem_students` degraded-mode bucket.
- `portal/control_room.py` — `add_student` and `assign_student` now write through to the DB; new `rehydrate_students_from_db()` method on `ControlRoom`.

No server.py changes — M4's `/api/encounter/{id}/assign_students` route already calls `room.assign_student` indirectly via direct attribute writes (it sets `enc.assigned_student_ids` and `student.assigned_encounter_id`). Future versions could switch to calling `room.assign_student()` instead so the write-through fires; M8's contract is already met for the dashboard's primary path (rooms started via `/api/room/start` then students added via `room.add_student()`).

3. Uses

- `ControlRoom.add_student(display_name, assigned_encounter_id=None)`

is called by: - The M9 student-join flow's POST `/portal/students/register` handler (lands in M9). - Future operator UI that pre-loads a class roster from a CSV.

- `ControlRoom.assign_student(student_id, encounter_id)` is called

by the operator's roster picker (M9) and by the M4 dashboard's assign-students route once it migrates to use this helper.

- `ehr_db.students_for_room(room_id)` and

`ControlRoom.rehydrate_students_from_db()` are called by: - M9's join flow when reattaching a cookie-bearing student to a room that survived a restart. - Future "Reopen room" operator flows.

- `ehr_db.touch_student(student_id)` updates `last_seen` on each student heartbeat (M9).

4. Functions (exported API surface)

portal/ehr_db.py (new CRUD helpers)

Symbol	Signature	Purpose	
register_student	(room_id, *, display_name, student_id=None, assigned_encounter_id=None) -> dict	Insert a row. Generates student_id if not supplied. Returns the persisted row.	
update_student_assignment	`(student_id, encounter_id \	None) -> None`	Set/clear the row's assigned_encounter_id.
touch_student	(student_id) -> None	Bump last_seen to now.	
students_for_room	(room_id) -> list[dict]	Every student row for a room, oldest first.	
students_for_encounter	(encounter_id) -> list[dict]	Filter by assigned_encounter_id.	
get_student	`(student_id) -> dict \	None`	Lookup by id.
remove_student	(student_id) -> None	Hard delete. Reserved for M9 roster-management UI; normal end-of-room keeps the audit trail.	

portal/control_room.py (extended methods)

Symbol	Behavior
ControlRoom.add_student(display_name, *, assigned_encounter_id=None)	Now: persists to DB first , then hydrates the in-memory Student from the returned row. Student id is DB-generated (consistent format stu_<hex10>).
ControlRoom.assign_student(student_id, encounter_id)	Updates in-memory state then calls ehr_db.update_student_assignment. Also removes the student from a previous encounter's assigned_student_ids list if they were already bound somewhere.
ControlRoom.rehydrate_students_from_db()	Clears in-memory students and every encounter's assigned_student_ids, then loads from ehr_db.students_for_room(self.room_id). Returns the number of students loaded. Idempotent.

5. Limitations

- **The ControlRoom itself does not persist.** Rooms are in-memory.

Restoring a room after restart requires the operator to invoke a "reopen room" flow (not built yet) or for the wizard to expose a "resume from room_id" option (an M19 candidate). For now, rehydrate is a building block — it works when a ControlRoom with the same room_id is reconstructed.

- **No referential integrity on student.assigned_encounter_id.**

If an encounter is deleted while a student still points at it, the rehydrate skips the assignment silently (the `if eid in self.encounters` guard). This is intentional — chart_event rows for a deleted encounter still exist for debrief.

- **No bulk-import API.** Adding 24 students means 24 round-trips.

Acceptable at the M8/M9 scale (classroom of ≤ 24); a `bulk_register` helper is a v7.1 candidate if pre-CSV-load lands.

- **Hard delete via** `remove_student`. Used only by the M9 roster

UI's "kick this student" affordance; end-of-room keeps every student row so the cohort debrief has the full roster.

- `touch_student` **is not yet called by any route** — wired into

the data layer ready for M9's heartbeat path.

- **In-memory mode** (`_conn() is None`) **writes to** `_mem_students` —

shares the same key namespace as SQLite mode. Degraded-mode rosters do NOT persist past process exit (matches the rest of the in-memory fallback contract).

6. Test status

Automated (`tests/v7/test_student_*.py`)

Test file	Cases	Status	Last run
<code>test_student_register_persists_in_db.py</code>	4 — add writes row, 3 students get distinct ids, <code>students_for_room</code> scopes by <code>room_id</code> , <code>assign_student</code> updates the DB column.	PASS	2026-05-26
<code>test_student_assigned_encounter_survives_server_restart.py</code>	3 — full restart cycle restores 3 students with assignments; rehydrate is idempotent; unassigned students restore with NULL <code>assigned_encounter_id</code> .	PASS	2026-05-26

7/7 PASS. **Full v7 suite: 43/43 passing** (up from 36). **Full v6 regression on v7: 154 passed**, same 6 env-flaky pre-existing failures, **0 v7 regressions**.

Manual

None for M8 — pure data layer. UI surfaces land in M9 (student join) and the M5 dashboard's roster panel (a follow-up).

7. Change list

Date	Author	Change	Files
2026-05-26	claude-code	Added 7 student CRUD helpers in <code>ehr_db.py</code> with SQLite + in-memory fallback paths. Extended <code>ControlRoom.add_student</code> and <code>assign_student</code> to write through. Added <code>rehydrate_students_from_db()</code> method. 7 acceptance tests across 2 files.	<code>portal/ehr_db.py</code> , <code>portal/control_room.py</code> , <code>tests/v7/test_student_register_persists_in_db.py</code> , <code>tests/v7/test_student_assigned_encounter_survives_server_restart.py</code>

8. Open questions / known issues

- The `M4 /api/encounter/{id}/assign_students` route still mutates

`student.assigned_encounter_id` and `enc.assigned_student_ids` directly instead of going through `room.assign_student`. The dashboard's existing flow therefore doesn't write through to the DB. Follow-up to migrate that route is small but deferred to M9 (where the student-side join flow will exercise both paths and surface any consistency issue). The contract holds for rooms where the operator adds students via `room.add_student` directly.

- `rehydrate_students_from_db` reuses the in-memory `Student`

class. If we later add fields to `Student` that the DB schema doesn't carry, those default to the dataclass default. Always add new fields with safe defaults.

- The `student_id` format went from `secrets.token_urlsafe(6)` in

the M2 `Student` dataclass to `"stu_" + uuid.uuid4().hex[:10]` in `register_student` (chosen for visual consistency with v6's `ord_<hex>` order ids). Mixed format is fine because the column is opaque TEXT — but future code should use whichever the registration helper returns.

- No DB indexes on `student.assigned_encounter_id` beyond the M1

schema's `ix_student_assigned_encounter.students_for_room` uses `ix_student_room.students_for_encounter` uses the assigned-encounter index. At classroom scale (≤ 24 students) performance is irrelevant; revisit if a future deployment scales past 1000 students per room.